Generics

```
// class
class Box<T> { }
// variable
Box<String> b = new Box<String>();
// method
public <T> void doStuff(T value) { }
```

Java being stupid

```
T obj = new T();          // error
obj instanceof T;         // error
T heh = (T) "asdf";       // no typechecking
var s = Stack<int>();     // no primitive types
private static T x;       // static no workey
Node<Number> s = new Node<Integer>; // error
class IThrowAnErrorBecauseWhyNot {
  void m(List<String> list) { }
  void m(List<Integer> list) { }
} // error because of type erasure & identifiability
```

Iterator

```
for (String s : stringList) { } // ==
for (Iterator<String> i = stringList.iterator();
    i.hasNext();) {
  String s = i.next();
}
```

Iterable

```
interface Iterable<T> {
  Iterator<T> iterator();
}
interface Iterator<E> {
  boolean hasNext();
  E next();
}
```

Comparable

TODO:
Type signature?

```
static <T extends Comparable> T doStuff(T a, T b) {
  // compareTo is possible with all types
  return a.compareTo("b") > 0 ? a : b;
}
static <T extends Comparable<T>> T doStuff(T a, T b)
{
  // compareTo is possible only with T
  return a.compareTo(b) > 0 ? a : b;
}
```

Example

```
class Graphic {
  public void draw() { }
}
class Rectangle extends Graphic { }
class Circle extends Graphic { }
class GraphicStack<T extends Graphic>
    extends Stack<T> {
  public void drawAll() {
    for (T item : this) item.draw();
  }
}
```

Bytecode

TODO:

Type Erasure

- Introduced because of "bAcKwArDs CoMpAtAbIlItY"
- No type information at runtime (Non-Reifiable Type)

```
class MyStack<T> {
  Entry<T> top;
  int push(T value) { }
}
// becomes
class MyStack {
  Entry top;
  int push(Object value) { }
}
/* with bounds */
class MyStack<T extends Comparable<T>> {
  Entry<T> top;
  int push(T value) { }
}
// becomes
class MyStack {
  Entry top;
  int push(Comparable value) { }
}
/* what happens at/before runtime */
MyStack<String> s = new MyStack<String>();
s.push("Eh");
String x = s.pop();
// becomes
MyStack s = new MyStack();
s.push("Eh");
String x = (String) s.pop();
```

Wildcards

```
void printGs(List<Graphic> gs) { }
List<Graphic> gs1 = new ArrayList<>();
printGs(gs1);   // ok
List<Circle> gs2 = new ArrayList<>();
printGs(gs2);   // error
/* solution */
void printGs(List<? extends Graphic> gs) { }
```

Generic variance

	Type	Compatible Type-Args	R	W
Invariance	C<T>	T	✓	✓
Covariance	C<? extends T>	T and sub-types	✓	✗
Contravariance	C<? super T>	T and basis-types	✗	✓
Bivariance	C<?>	All	✗	✗

Invariance

```
static <T> void move(Stack<T>, from, Stack<T> to) {
  while(!from.isEmpty()) to.push(from.pop());
}
var rectS = new Stack<Rectangle>();
var graphS = new Stack<Graphic>();
graphS.push(new Rectangle());          // ok
move(rectS, graphS);                   // error
Stack<Object> objS = graphS;           // error
/* anotherone */
String[] strA = new String[10];
Object[] objA = strA;                  // ok
objA[0] = Integer.valueOf(2);          // runtime err
```

TODO:

```
// compiles?
List<Graphic> list = new
ArrayList<Rectangle>();
// compiles?
Rectangle[] rectangleStack = new
Rectangle[10];
Graphic[] graphicStack = new Graphic[10];
graphicStack = rectangleStack;
// compiles?
Stack<Rectangle> rectangleStack = new
Stack<>();
Stack<Graphic> graphicStack = new Stack<>();
graphicStack = rectangleStack;
// compiles?
Stack<Rectangle> rectangleStack = new
Stack<>();
Stack<Graphic> graphicStack = new Stack<>();
for (Rectangle r : rectangleStack) {
  graphicStack.push(r);
}
```

Covariance

```
public class Stack<E> {
  public void pushAll(Iterable<? extends E> src) { }
}
Stack<? extends Graphic> graphS;
graphS = new Stack<Rectangle>();     // ok
graphS = new Stack<Circle>();        // ok
graphS = new Stack<Object>();        // error (duh)
/* read only */
Stack<? extends Graphic> stack = new
  Stack<Rectangle>();
stack.push(new Graphic());           // error
stack.push(new Rectangle());         // error
stack.push(new Triangle());          // error
```

Contravariance

```
static <T> void addToC(List<? super T> list, T e) {
    list.add(e);
}
addToC(new ArrayList<Number>(), 3); // ok
addToC(new ArrayList<Object>(), 3); // ok
/* shenanigans */
Stack<? super Graphic> s = new Stack<Object>();
s.add(new Graphic());                // ok
s.add(new Rectangle());              // ok
Graphic g = s.pop();                 // error
Object g = s.pop();                  // ok
```

Bivariance

```
static void printList(List<?> list) {
  for (Object elem : list)
    System.out.print(elem + " ");
}
/* more shenanigans */
static void appendNewObject(List<?> list) {
  list.add(new Object());            // error
}
```

Producer / Consumer

TODO:

Misc

```
<T extends Comparable & Collection> // multiple type
bounds
```

Generics stream tomfoolery

```
List<? extends Media> mediaList;
public <S extends T> List<S>
  filterBySubtype(Class<S> subtype) {
    return mediaList.stream()
            .filter(subtype::isInstance)
            .map(subtype::cast)
            .collect(Collectors.toList());
}
```

Annotations

- Metadata
- Not actually part of the code

TODO:
slides 10

- @Override
- @Test
- @Json…

Defining

```
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
public @interface Author {
  String name();
  String date() default "N/A";
}

@Author(name = "UwU", date = "idon'tvalidateinput")
public class WeeOoo { }
```

Reflection

- Information of Class
  - Methods
  - Attributes
  - Parent class
  - Implemented interfaces
- Calling methods possible
- Usages
  - Debugger
  - Serialization
  - Frameworks
  - Remote Procedure Call
  - ORM

```
// all of these `throws SecurityException`
public Field[] getDeclaredFields()
public Method[] getDeclaredMethods()
public Constructor<?>[] getDeclaredConstructors()

System.out.println(dump(new Student("a", "b"), 0));
// {
//   firstName = a
//   lastName = b
// }
Class c = "s".getClass(); // java.lang.String
```

TODO:
Diagram (slides 26)

Method

```
// @Target(ElementType.METHOD)

public String getName()
public boolean isAnnotationPresent(
  Class<? extends Annotation> annotationClass)
public Object invoke(Object obj, Object... args)
  throws IllegalAccessException,
         IllegalArgumentException,
         InvocationTargetException

public class Profiler {
  public static void main(String[] args) {
    var testFunctions = new ProfileFunctions();
    int[] array = {5, 2, 8, 1, 93, 3, 33, 1, 333};
    var methods = ProfileFunctions.class
                    .getDeclaredMethods();
    for (var m : methods) {
      if(m.isAnnotationPresent(Profile.class)) {
        m.setAccessible(true); // best practice or
sth, idk, i write code in actually good languages
        Profiler.profileMethod(testFunctions, m,
          new Object[] {array});
      }
    }
  }
}
```

Class

TODO:

```
// @Target(ElementType.TYPE)

getDeclaredConstructor().newInstance()
...
```

Attribute

TODO:

```
// @Target(ElementType.FIELD)

...
```

Validation

```
@Min(value = 18, message = "Age must be ≥ 18")
@Max(value = 99, message = "Age must be ≤ 99")
private int age;
@NotNull(message = "Name cannot be null")
private String name;
```

Algorithms

- Brute-force
- Greedy
  - Take best option each step (optimize locally)
  - Fast
- Backtracking
  - Trial and error
  - Best overall option
  - Higher compute costs
- Divide and conquer
  - Binary search
- Dynamic programming
  - Recursion optimization?
  - Tabulation

Big O Notation

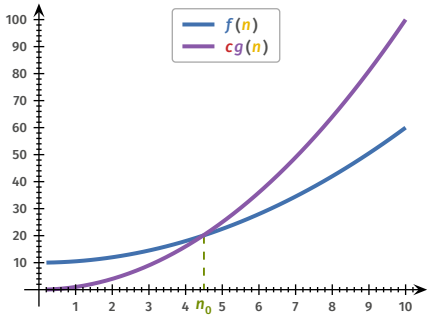
- Worst case scenario
- Atomic operations = constant time
- Runtime measured as sum of primitive operations

Notation

$$\overbrace{f}^{\text{Algorithm}}\left(\underbrace{n}_{\text{Size of problem}}\right) = \text{Number of steps}$$

g = complexity class

TODO:
diagram $f \le cg$



$f(n)$ is $O(g(n))$ if $c > 0 \wedge n_0 \ge 1$

$f(n) \le cg(n)$ for $n \ge n_0$

f in order g:

Let $f, g : \mathbb{N} \to \mathbb{N}$

$n_0, c \in \mathbb{N} \wedge \forall n \ge n_0 : f(n) \le c \cdot g(n)$

$\Rightarrow f \in O(g) \Leftrightarrow f = O(g)$

Big O

$$\overbrace{O}^{\text{Big O}}\left(\underbrace{n}_{\text{Size of problem}}\right) = \text{Complexity class}$$

Rules

If $f(n)$ is polynomial of degree d, then $f(n) \in O(n^d)$

- ignore lower powers
- ignore constants

Use most optimal function (lowest power)

- $2n$ is $O(n)$ better than $2n$ is $O(n^2)$

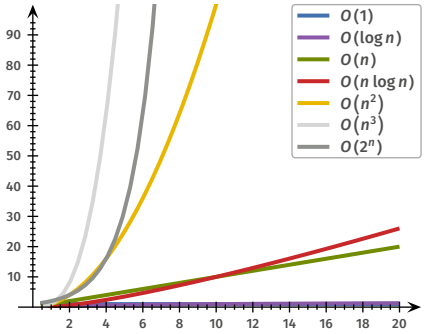
Simplify as far as possible

- $3n + 5$ is $O(n)$ instead of $3n + 5$ is $O(3n)$

Complexity classes

Name	Class	Example
Constant	1	Array read
Logarithmic	$\log n$	Binary search
Linear	n	Linear search
Log-linear	$n \log n$	Merge+Quick sort
Quadratic	n^2	Bubble+Insertion sort
Cubic	n^3	Solve equation
Polynomial	n^k	Various algorithms

Name	***Class***	***Example***
Exponential	2^n	Calculate all subsets
Factorial	$n!$	Calculate all permutations



TODO:
Counting primitive operations (slides 55)